EXPERIMENT NO. 13

Implement Exception Handling and User-Defined Exceptions

PART B – TO BE COMPLETED BY STUDENTS

Roll No.: C-60

Name: Avdhut Parish Madhavi

Class / Division: SE-C

Batch: C2

Date of Experiment: _____

Date of Submission: _____

Grade: _____

B.1 Document Created by the Student

Change the validation condition and test both valid and invalid values:

The validation condition is changed to accept patient ages from 1 to 100 years. The program is tested with a valid age of 25 and an invalid age of 150. For age 25, the program displays the accepted age. For age 150, the user-defined exception is thrown and caught, and an informative error message is displayed. The finally block displays the completion message in both cases.

B.2 Student Program / Output

Paste/write the final program and attach or reproduce the observed output below:

Program: ExceptionDemo.java

```
class InvalidPatientAgeException extends Exception {
   InvalidPatientAgeException(String message) {
      super(message);
   }
}

public class ExceptionDemo {
   static void validateAge(int age)
         throws InvalidPatientAgeException {
      if (age < 1 || age > 100) {
         throw new InvalidPatientAgeException(
               "Invalid patient age. Age must be from 1 to 100.");
```

```
        }
        System.out.println("Age accepted: " + age);
    }

    public static void main(String[] args) {
        int age = 25;  // Change to 150 to test invalid age

        try {
            validateAge(age);
        } catch (InvalidPatientAgeException e) {
            System.out.println("Handled: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Other error: " + e.getMessage());
        } finally {
            System.out.println("Validation completed.");
        }
    }
}
```

Output 1 – Valid age (age = 25):

```
C:\Java\JAVA PRACTICALS>java ExceptionDemo.java
Age accepted: 25
Validation completed.
```

Output 2 – Invalid age (change age to 150):

```
C:\Java\JAVA PRACTICALS>java ExceptionDemo.java
Handled: Invalid patient age. Age must be from 1 to 100.
Validation completed.
```

B.3 Observations and Learning

• What is the difference between throw and throws?

throw is used to explicitly raise an exception at a particular point in the program. throws is written in a method declaration to indicate that the method may pass an exception to its caller.

• Why are custom exceptions useful?

Custom exceptions represent application-specific error conditions and allow meaningful error messages to be displayed, such as an invalid patient age.

• **What is the purpose of finally?**

The finally block normally executes after try/catch processing, whether an exception occurs or not. It is used for completion actions and cleanup.

Student observation / learning:

I learned to validate a patient's age using a user-defined exception. The throw statement raises the exception, throws declares it in the method signature, catch handles it, and finally displays the completion message for both valid and invalid values.

B.4 Conclusion

Validation of patient records is explicitly included in the syllabus self-learning context.

Student conclusion:

The experiment successfully demonstrates exception handling in Java using try, multiple catch blocks, finally, throw, throws, and a user-defined exception. Valid and invalid patient ages are tested, and an informative message is displayed when the age is outside the accepted range.

B.5 Questions of Curiosity

1. What is the difference between throw and throws?

throw explicitly raises an exception, whereas throws declares in a method signature the exception that may be passed to the caller.

2. Why are user-defined exceptions useful?

They represent application-specific errors and help provide meaningful messages for invalid conditions.

3. What is the purpose of finally?

The finally block normally executes after try/catch processing, whether an exception occurs or not.